



Misión de Emergencia: ¡El Rescate del OECISEY-1! – Primera Etapa

¡Atención, equipo de computistas! Tenemos una situación crítica en el espacio profundo. Nuestro satélite de comunicaciones OECISEY-1, encargado de retransmitir los datos vitales de la Tierra, ha sufrido una catástrofe catastrófica.

Debido a una tormenta solar inesperada, dos de los microprocesadores principales de la unidad de cómputo han quedado carbonizados. La lógica original del procesador del OECISEY-1 era una maravilla de la ingeniería, pero ahora, con hardware limitado y una arquitectura degradada, el sistema está al borde del colapso total.

El satélite aún puede operar, pero solo si logramos "hackear" su propio ensamblador. Los microprocesadores que sobrevivieron no entienden el juego de instrucciones original; el mapa de registros ha cambiado y las rutas de memoria se han vuelto erráticas.

Tu misión, si decides aceptarla, es:

Reconfigurar el Ensamblador: Debes adaptar el ensamblador del OECISEY-1 para que pueda traducir instrucciones hacia las unidades de procesamiento que todavía funcionan. Si el código no es exacto, el satélite se quedará "mudo" en órbita.

El Protocolo de Supervivencia: Como el procesador está dañado, el satélite requiere que implementes técnicas de acceso a memoria con desplazamiento para evitar las zonas "quemadas" de la placa base (los sectores de memoria dañados).

Descifrado de Emergencia: El OECISEY-1 está enviando sus datos de telemetría cifrados para protegerlos de posibles interferencias. Deberás escribir una rutina en ensamblador que utilice las limitadas instrucciones de la ALU para descifrar el mensaje de auxilio y enviarlo de vuelta a la base.

Si no logran estabilizar el código:

- El OECISEY-1 dejará de funcionar y perderemos la conexión con él.
- El mensaje de emergencia se perderá en el vacío del espacio.
- ¡Y lo más importante, su nota del semestre podría correr la misma suerte que los microprocesadores del satélite!

¿Tienen lo que se necesita para reprogramar un satélite a la deriva y salvar la misión OECISEY-1, o dejaremos que el satélite se convierta en basura espacial?

Especificaciones Técnicas para los Técnicos de Vuelo:

- Lenguaje: C (El lenguaje que mantiene el universo unido).
- Reto: El uso de MRMOVQ y RMMOVQ con desplazamiento es obligatorio para esquivar los sectores de memoria que están "en llamas".

El objetivo de este proyecto es profundizar en el funcionamiento interno de una Unidad Central de Procesamiento (CPU) mediante el diseño de un emulador de ciclo único en software. Los estudiantes deberán modelar el comportamiento del hardware a través de las cinco etapas clásicas de la arquitectura de computadores: Fetch (Captación), Decode (Decodificación), Execute (Ejecución), Memory (Memoria) y Write-Back (Escritura en Registros). Para este proyecto. trabajaremos con una variante propietaria y modificada de la arquitectura didáctica Y86-64, Esta arquitectura utiliza un formato de instrucción de longitud fija, lo que simplifica el flujo de control y fuerza el análisis de manipulación de bits.

Especificación del Formato de Instrucción

Toda instrucción en nuestra CPU ocupa exactamente 8 bytes (64 bits). Para facilitar la etapa de Fetch y Decode en C, cada instrucción debe ser mapeada y decodificada utilizando la siguiente estructura de campos de bits:

```
typedef struct {
    uint8_t icode;           // Código de operación (4 bits)
    uint8_t ifun;            // Función específica (4 bits)
    uint8_t rA;              // Registro fuente A (4 bits)
    uint8_t rB;              // Registro fuente/destino B (4 bits)
    uint64_t valC;           // Valor inmediato o dirección (48 bits)
} Instruction;
```

Set de Instrucciones y Matriz de Control Modificada

A continuación, se detalla la tabla de codificación oficial para este proyecto. Nota Crítica: Los códigos de operación (opcodes) no corresponden al estándar de la literatura clásica. El emulador debe ceñirse estrictamente a los siguientes valores:

Instrucción	icode	ifun	Descripción Funcional
HALT	0xE	0x0	Detiene inmediatamente la ejecución del emulador.
NOP	0xF	0x0	No realiza ninguna operación; avanza el PC.
RRMVQ	0x5	0x0	Transfiere el valor del registro rA al registro rB.

Instrucción	icode	ifun	Descripción Funcional
IRMOVQ	0x1	0x0	Carga el valor inmediato (valC) en el registro destino rB.
OPQ	0xA	0x0 - 0x3	Operaciones de la ALU (Ver subtabla de funciones).
JXX	0x3	0x0 - 0x2	Salto condicionales (0x0: JMP, 0x1: JL, 0x2: JE).
PUSH	0x8	0x0	Resta 8 a %rsp y guarda el valor de rA en la memoria.
POP	0x9	0x0	Lee el valor de la pila en rA e incrementa %rsp en 8.

Funciones de la ALU (icode = 0xA)

- ifun = 0x2 (ADDQ): Realiza la suma. Modificación de destino: $\$rA \leftarrow rA + rB\$$.
- ifun = 0x3 (SUBQ): Realiza la resta. Operación: $\$rB \leftarrow rB - rA\$$.
- ifun = 0x0 (ANDQ): Operación lógica AND bit a bit entre rB y rA.
- ifun = 0x1 (XORQ): Operación lógica XOR bit a bit entre rB y rA.

Sistema de Banderas (Flags) y Control de Excepciones

El procesador debe actualizar de manera precisa los registros de estado tras cada instrucción aritmética o acceso a memoria:

Banderas Aritméticas (ALU Flags)

- **ZF (Zero Flag):** Se activa si el resultado de la ALU es idéntico a cero.
- **SF (Sign Flag):**** Se activa si el resultado es un número matemáticamente par (bit menos significativo en 0).
- **OF (Overflow Flag):** Se activa ante desbordamientos de enteros con signo de 64 bits.

Banderas de Estado del Sistema (Registro STAT)

Si la CPU detecta una anomalía en el flujo, debe guardar el estado, levantar una excepción en el registro STAT y detener el ciclo:

- **STAT_HLT (0x1):** Apagado normal del sistema vía instrucción HALT.
- **STAT_INS (0x2):** Instrucción ilegal. Se activa si se decodifica un icode o ifun inexistente.
- **STAT_AEX (0x3):** Error de alineación. Se dispara si el PC o el puntero de pila (%rsp) no apuntan a una dirección múltiplo exacto de 8 bytes durante una lectura/escritura.
- **STAT_ADR (0x4):** Dirección inválida. Intento de acceso fuera del límite de la memoria simulada

Mapa de Registros "Reducido"

Usaremos 8 registros de uso general (R0-R7). Esto ahorra energía (y transistores en tu código C).

- **R0 (ZERO):** Siempre vale 0 (útil para despejar registros).
- **R1-R5:** Registros de datos para operaciones.
- **R6 (SP):** Stack Pointer (obligatorio para funciones).
- **R7 (PC):** Program Counter.

Instrucciones Exclusivas de Rescate

Para que el rescate del satélite sea un éxito, incluimos estas instrucciones específicas:

- **XORR Ra, Rb:** La herramienta principal de descifrado.
- **LDD Ra, D(Rb):** Load Data. Carga el valor de [Rb + D] en Ra. (¡Clave para evitar zonas dañadas!)
- **STD Ra, D(Rb):** Store Data. Guarda el valor de Ra en [Rb + D].
- **JNZ D:** Jump if Not Zero. Esencial para los bucles de descifrado.

Características/Requerimientos del Entregable (Software)

- **Grupo de 3 estudiantes, ni más ni menos, no se aceptaran proyectos individuales**
- El programa (software) se debe implementar en C.
- La entrada al programa es un código en lenguaje ensamblador el cual debe traducir a código de máquina. Para ello debe permitir entrada por teclado y archivo, debe aceptar archivos con extensión ".asm", el resultado debe generar un archivo similar a un ".bin" pero solo texto con extensión ".txt".
- Se debe construir la tabla de símbolos (Etiquetas y Direcciones) y mostrarla en el comienzo del archivo ".txt".
- Generar el código objeto (binario) y realizar la resolución de direcciones.
- El ensamblador desarrollado debe calcular automáticamente el Offset necesario, donde $\text{Offset} = (\text{Dirección de Etiqueta}) - \text{PC}$.
- La dirección de inicio del código de máquina son los 32bits menos significativos del resultado de la suma de las cédulas de los integrantes del equipo (colocar el cálculo y procedimiento).
- En la parte final del archivo ".txt" debe mostrar el contenido final de los registros incluyendo el PSR
- Mostrar el resultado de la traducción en tres columnas
- Dirección | Código de máquina en hexadecimal | # instrucción ensamblador
- Generar Log Requisitos:
 - **Generación Continua:** El emulador debe abrir un archivo de texto llamado trace.log al iniciar la ejecución y cerrarlo al finalizar (ya sea por HALT o por una excepción de estado).
 - **Formato de Registro:** Por cada ciclo de reloj completado, el simulador debe escribir una línea en el archivo con el siguiente formato exacto:
 - PC: 0x[VALOR_PC] | R0: 0x[VAL] R1: 0x[VAL] R2: 0x[VAL] R3: 0x[VAL] R4: 0x[VAL] R5: 0x[VAL] R6: 0x[VAL] R7: 0x[VAL] | STAT: 0x[VAL]
 - **Sincronización:** El registro debe reflejar el estado de los registros antes de que la instrucción del ciclo actual modifique su valor.
 - **Uso de la Traza:** Este archivo será la única herramienta válida para la depuración y la única prueba aceptada de que su procesador está esquivando correctamente las zonas de memoria dañadas.

Correo a enviar y adjuntar lo siguiente

- Código fuente de su programa bien documentado, se penalizara fuertemente el abuso de IA, su código tiene que ser compilado con gcc en linux, el archivo identificarlo como "grupo-NOMBRE-cod-f1.cc"
- Archivo ejecutable en un entorno linux identificado como "grupo-NOMBRE-exe-f1"
- Un informe breve explicando como llegaron a su propuesta de software, incluir cuales fueron sus retos y como lo superaron, el informe identificarlo como "grupo-NOMBRE-informe-f1.pdf"
- El asunto del correo es "proyecto-grupo-NOMBRE-fase-1"
- **FECHA DE ENTREGA: 15 de Julio**